DBMS PROJECT-BASED LEARNING • REVIEW 2

Car Rental Reservation & Vehicle Return Management System

Database Implementation — Schema, Constraints, Queries & Live Prototype

Presented by: Koneri Mithul | Roll Number(s)-25WU0102129 | Class & Section-AIML PANTHERS

Guide: Dr. Kiran Mayee Adavala | Department of Computer Science



PROGRESS

From Design to a Working Database

Review 1 — Completed

- ✓ Problem statement, scope & objectives
- ✓ Users & functional requirements
- ✓ ER diagram — 11 entities
- ✓ Initial relational schema

Review 2 — New This Phase

- ✓ Schema normalized to 3NF, with proof
- ✓ Full MySQL DDL: PK/FK/CHECK/UNIQUE/triggers
- ✓ 60+ rows of realistic sample data
- ✓ Joins, subqueries, aggregates & 6 views
- ✓ Transactions (COMMIT/ROLLBACK) & indexing
- ✓ Live, tested database — zero errors

NORMALIZATION

1NF → 2NF → 3NF, With a Real Example

1NF

Every column atomic, no repeating groups

Vehicle has one make, one model, one status per row — no comma-separated lists.

2NF

Non-key attributes depend on the WHOLE key

All 11 tables use a single-column surrogate key, so 2NF holds automatically once 1NF does.

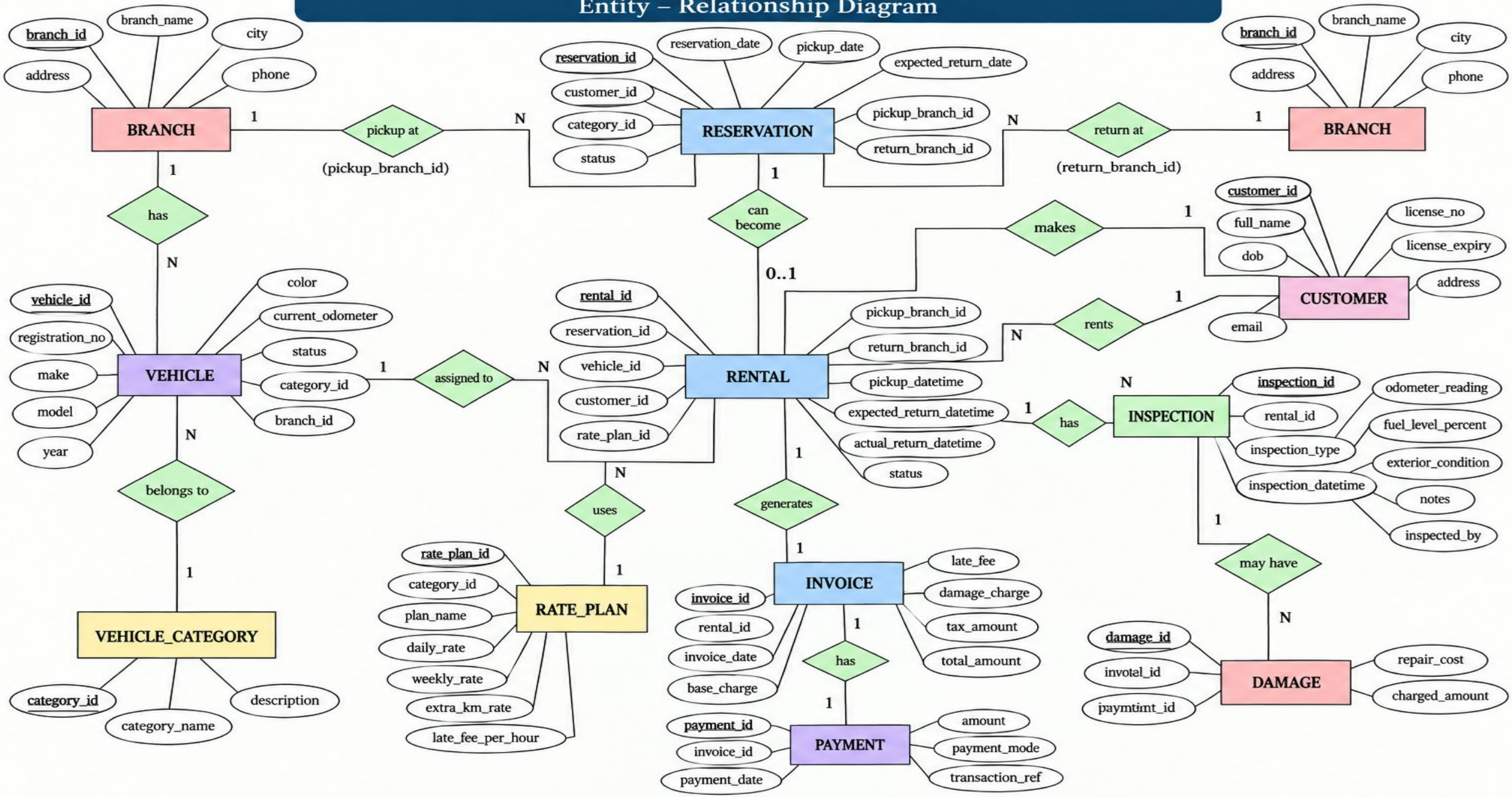
3NF

No attribute depends on another non-key attribute

Vehicle stores category_id only — NOT category_name. category_name depends on category_id, not on vehicle_id, so it stays in VehicleCategory.

Car Rental Reservation & Vehicle Return Management System

Entity – Relationship Diagram



NORMALIZATION

Key Functional Dependencies

```
Vehicle:  vehicle_id → registration_no, category_id, branch_id, make, model, status

Customer: customer_id → full_name, phone, email, license_no, license_expiry

Rental:   rental_id → vehicle_id, customer_id, rate_plan_id, pickup_datetime, actual_return_datetime

Invoice:  invoice_id → rental_id, base_charge, extra_km_charge, late_fee, total_amount

Payment:  payment_id → invoice_id, amount, payment_mode, payment_date
```

Every determinant above is the WHOLE primary key, and everything it determines depends on it directly — never on another non-key column. That is 3NF, applied literally.

IMPLEMENTATION

Relational Schema — All 11 Tables

```
Branch(branch_id PK, branch_name, city, address, phone)

VehicleCategory(category_id PK, category_name UK, description)

Vehicle(vehicle_id PK, registration_no UK, category_id FK, branch_id FK, ..., status)

Customer(customer_id PK, phone UK, email UK, license_no UK, license_expiry, ...)

RatePlan(rate_plan_id PK, category_id FK, daily_rate, extra_km_rate, late_fee_per_hour)

Reservation(reservation_id PK, customer_id FK, category_id FK, pickup/return_branch_id FK, ...)

Rental(rental_id PK, reservation_id FK NULL, vehicle_id FK, customer_id FK, rate_plan_id FK, ...)

Inspection(inspection_id PK, rental_id FK, inspection_type, odometer_reading, fuel_level_percent)

Damage(damage_id PK, rental_id FK, inspection_id FK NULL, repair_cost, charged_amount)

Invoice(invoice_id PK, rental_id FK UK, base_charge, extra_km_charge, late_fee, total_amount)

Payment(payment_id PK, invoice_id FK, amount, payment_mode, transaction_ref)
```

DOCUMENTATION

Data Dictionary — Rental (Representative)

Column	Type	Key	Null	Default	Notes
rental_id	INT	PK	NOT NULL	AUTO_INC	
reservation_id	INT	FK	NULL		Nullable — walk-ins
vehicle_id	INT	FK	NOT NULL		
customer_id	INT	FK	NOT NULL		
rate_plan_id	INT	FK	NOT NULL		
pickup_datetime	DATETIME		NOT NULL		
actual_return_datetime	DATETIME		NULL		CHECK > pickup
status	VARCHAR(15)		NOT NULL	'Active'	CHECK IN (...)

Full data dictionary for all 11 tables is in the project report — shown here is one representative table.

IMPLEMENTATION

DDL & Integrity Constraints

PRIMARY KEY	Every table — AUTO_INCREMENT surrogate key
FOREIGN KEY	Every relationship, with explicit ON DELETE/UPDATE
UNIQUE	registration_no, phone, email, license_no, Invoice.rental_id
CHECK	Value ranges: year, odometer ≥ 0, fuel 0–100%, charges ≥ 0, date ordering
NOT NULL / DEFAULT	Required fields; status defaults ('Available', 'Active', 'Booked')
TRIGGER	5 triggers — cross-row rules + automatic vehicle status sync

IMPLEMENTATION

Realistic Sample Data Loaded

4 Branches	4 Categories	12 Vehicles	9 Customers	5 Rate Plans	10 Reservations
9 Rentals	17 Inspections	3 Damages	7 Invoices	8 Payments	

Deliberately includes: available / rented / maintenance vehicles • an active rental • an overdue rental • a rental with damage • an invoice paid in two partial installments

QUERYING THE DATABASE

Queries, Subqueries, Aggregation & Views

Joins	Customer rental history, Invoice+Payment, Damage+Vehicle+Customer
Subqueries	Customers/vehicles above-average rental count, branches above-average revenue
Aggregation	COUNT/SUM/AVG/MIN/MAX with GROUP BY — revenue, utilization, damage cost
Views (6)	Fleet Availability, Overdue Rentals, Customer History, Branch Revenue, Utilization, Damage Report

DATA INTEGRITY

Business Rules — Tested & Confirmed

No overlapping rentals	Trigger	✓ Confirmed: rejected with overlap error
Valid driving licence	Trigger	✓ Confirmed: rejected with expiry error
Return after pickup	CHECK	✓ Confirmed: rejected by CHECK constraint
Valid odometer / fuel	CHECK + Trigger	✓ Confirmed: rejected with odometer error
Non-negative charges	CHECK	✓ Confirmed: rejected by CHECK constraint



Live Database Prototype

Demo flow: Customer → Reservation → Rental → Inspection → Damage → Invoice → Payment

- Tables, keys & relationships (DESCRIBE / SHOW)
- INSERT, SELECT, JOIN, subquery, view — live
- Constraint & trigger rejections — shown failing on purpose
- Transaction COMMIT and ROLLBACK, verified after each

Achieved: a normalized, constraint-enforced, tested MySQL database ready for the Review 3 application layer.

Thank You — Questions & Discussion